Relational Database Management System Implementation System Documentation

Allegra Harris, Charlie Mei, and Sarah Green

1. Introduction

The purpose of the Table Management System is to provide a flexible and efficient platform for managing tabular data within a relational database. It allows users to define and manipulate tables, define primary and foreign keys, and perform various operations such as inserting, updating, and querying data. The system is designed to be user-friendly while offering robust functionality for handling structured data.

2. System Overview

2.1. Architecture

The Table Management System is built on a modular and extensible architecture. It consists of a core table management module responsible for handling table creation, modification, and data manipulation. The system leverages the following key components:

Table Class: The fundamental building block, encapsulating the structure and behavior of a table. It maintains information about columns, primary and foreign keys, and column data.

Database Module: A global variable ‘databases’ serves as the database repository, storing all tables created within the system. This module facilitates the organization and retrieval of tables.

Exception Handling: The system includes custom exception classes (Invalid_Type, Syntax_Error, Duplicate_Item, Keyword_Used, Not_Exist, Unsupported_Functionality) to handle various error scenarios gracefully.

External Libraries: The system utilizes the external libraries ‘sqlparse’ for user input parsing, ‘re’ for regular expressions, and ‘tabulate’ for tabular data formatting.

2.2. Usage

2.2.1. Table Management

Users can define tables by specifying column names, data types, and primary/foreign keys. Tables must have a primary key defined, and foreign key must be defined if it exists as a primary key in another table. The system ensures the integrity of keys and columns during table creation, and allows for tuple insertion.

2.2.2. System Interaction

Program will be executed by typing the following command in Database Project 3 Code folder:

```
-> python3 main.py
```

Once the program is executed, interaction with the system occurs via a command-line interface. Users can define tables, add attributes (columns, primary keys, foreign keys), insert data, and perform various operations using the provided commands.

2.2.3. Preloaded Queries

Throughout this documentation, we will reference the following pre-loaded tables, rel1, rel2, rel3, and rel4. Each table contains col1 and col2 (INTs). col1 is the primary key of each table.
Rel1 INSERT (1,1), (2,2), …, (999, 999), (1000,1000)
Rel2 INSERT (1,1), (2,1), …, (999, 1), (1000,1)
Rel3 INSERT (1,1), (2,2), …, (9999, 9999), (10000,10000)
Rel4 INSERT (1,1), (2,1), …, (9999, 1), (10000,1)

3. Features

3.1. Input Parsing

Our system utilizes sqlparse to format user input. It tokenizes and filters the input for further processing. Here is an example of valid statement and its filtered form:

```
-> SELECT * FROM employees;
-> [‘SELECT’, ’*’, ’FROM’, ’employees’, ’;’]
```

3.2. Table Creation and Manipulation

To create a new table, our system utilizes the Table class and its methods. Users should specify the column names and keys for each table they create. Each column in a table should be defined as either an integer or string type. Additionally, primary and foreign keys should be identified. Single and multiple-attribute primary keys are supported. Here's an example of creating two tables with columns using the CREATE keyword:

```
-> CREATE TABLE departments (dep_id INT, dep_name STRING, primary key
(dep_id));
-> CREATE TABLE employees (emp_id INT, emp_name STRING, dep_id INT, primary key
(emp_id), foreign key (dep_id) references departments(dep_id));
```

Once a table is defined, users can insert tuples into the table using the INSERT keyword. Users must insert values into all columns as the system does not support null values.

```
-> INSERT INTO departments VALUES (1,'chem'), (2,'math'), (3,'cs'),
(4,'physics'), (5,'biology');
```

N.B. If column names are specified in an INSERT, all columns must be specified as nullable values are not supported

3.3. SQL Keywords

The system supports various SQL keywords such as CREATE, SHOW, DESCRIBE, INSERT, SELECT, WHERE, and JOIN. These keywords handle table creation, display, description and tuple insertion. All of the keywords are case insensitive.

3.4. Aggregate Operators

The aggregation operators MAX, MIN, AVG and SUM are supported in the system. Users can perform aggregations on specific columns to derive summary statistics:

```
-> SELECT max(emp_id) FROM employees;
```

N.B. MIN, AVG and SUM were added after the demo

3.5. Two-Clause Logical Conditions

The system supports logical conjunction (‘AND’) and disjunction (‘OR’) in queries. Users can compose queries with conditions involving multiple clauses, enhancing the flexibility of data retrieval:

```
-> SELECT * FROM employees WHERE emp_id > 1000 AND dep_id = 3;
```

3.6. Two-Table Equal-Join

The system supports joining two tables based on equal conditions. Users can define foreign keys and perform join operations on related columns. The system enforces primary key uniqueness and foreign key referential integrity.

```
-> SELECT * FROM employees JOIN departments ON employees.dep_id =
departments.dep_id WHERE employees.emp_name='sarah green' OR
departments.dep_name= 'cs';
```

3.7. Execution Engine

3.7.1. Data Definition Language

If the user inputs a statement using Data Definition Language (DDL), the system will imitate the MySQL output indicating how many rows are affected during the process.

3.7.2. Query Processing Time

In the same manner as MySQL, the query processing time only measures the time that a query takes to execute, but does not include the time it takes to print the table. The following query takes about 15 seconds to execute but 16 seconds to print (cartesian product with 1,000,000 tuples):

```
-> SELECT * FROM rel1 join rel2 on rel1.col1 = rel1.col1;
```

4. Optimization

4.1. Equal-Join from the Same Table

If the two statements for the join condition are from the same table then the cartesian product of the two tables is applied. If the column names are different then apply the 'WHERE' statement to reduce the size of one of the tables and then apply the cartesian product.

```
-> SELECT * FROM rel1 JOIN rel2 ON rel1.col1 = rel1.col1;
-> SELECT * FROM rel1 JOIN rel2 ON rel1.col1 = rel1.col2;
```

The above two statements will not call nested loop or merge scan functions. Instead, they will call the cartesian product to simply run a double loop to join each tuple from rel1 (or all the tuples that meet the condition) to each tuple from rel2.

N.B. This is an optimization that was added after the demo

4.2. Equal-Join on Primary Keys

If both the columns in a join are from each table's single primary key, then our system uses the primary key to access the indexing structure from the both tables. In this case, neither nested loop or merge scan will be executed. An example of this would be:

```
-> SELECT * FROM rel1 JOIN rel2 ON rel1.col1 = rel2.col1;
```

4.3. Sort-Merge vs. Nested-Loop Join Selection

The system is optimized to execute a Sort-Merge join when the tables being joined are close in size. It is optimized to execute a Nested-Loop join when sizeof(table1) > 1.5 * sizeof(table2).

For all operations requiring Nested-Loop joining for two tables, the table with less tuples is the outer loop.

```
-> SELECT * from rel3 join rel4 on rel3.col1 = rel4.col1 where rel4.col1 <=
100;
```

The query above will have rel4 be the outer relation and rel3 as the inner relation during the joining process because the WHERE clause reduces the size of rel4 from 10,000 to 100.

4.4. Boolean Optimization

Boolean optimization in WHERE clauses obey the following rules:

If a query is evaluating a clause comparing identical columns, then the system returns either the entire table or empty table. This rule also applies to conjunctive and disjunctive statements so iterating through the table is not necessary.

```
-> SELECT * FROM rel1 WHERE col1 = col1;
-> SELECT * FROM rel1 WHERE col1 != col1;
```

In the examples above, the first statement should return the entire table while the second one should return an empty table.

For the single-clause statements, if the column is the single primary key and the operator is '=', then apply the indexing structure to directly access the tuple.

```
-> SELECT * FROM rel1 WHERE col1 = 1;
```

For the conjunctive two-clause statements, we apply the single-clause statement first to the column that is part of the primary key before executing the second condition. For the disjunctive two-clause statements, apply both conditions and then take the union of the output.

4.5. Boolean Optimization with Join: Single Condition

WHERE clauses are always executed before join as long as the value being evaluated is constant. If the value is variable, this is not possible. Here are some examples:

```
-> SELECT * FROM rel1 JOIN rel2 on rel1.col1 = rel2.col1 WHERE rel1.col1 <=
100;
```
(Constant evaluated before join)

```
-> SELECT * FROM rel1 JOIN rel2 on rel1.col1 = rel2.col1 WHERE rel1.col1 !=
rel1.col1;
```
(Variable evaluated during join)

4.6. Boolean Optimization with Join: Double Condition

For execution involving double loop iteration, such as nested loop, cartesian product, or primary key equal join, the table with a smaller number of tuples will be on the outer loop.

```
-> SELECT * FROM rel1 join rel3 on rel1.col1 = rel3.col1;
```

In this case, primary key equal join will be executed and rel1 will be the outer relation. As a result we only need to look at 1,000 keys in rel1 compared to 10,000 keys in rel3 if we have rel3 as the outer relation.

If the clause is conjunctive, each clause can be executed before the join as long as the value is constant or a column name from the same table. Here is an example:

```
-> SELECT * FROM rel1 JOIN rel2 on rel1.col1 = rel2.col1 WHERE rel1.col1 <= 100
AND rel2.col1 <= 100;
```

Notice both conditions can be executed even though they are from different tables.

If the clause is disjunctive, the clause can only be applied if the above conditions are satisfied and both inputs are from the same table.

```
-> SELECT * FROM rel1 JOIN rel2 on rel1.col1 = rel2.col1 WHERE rel1.col1 <= 100
OR rel1.col1 >= 900;
```

The WHERE clause will be executed again after the table is joined (applying rules in Section 4.4) to verify the output (notice some of the where-clauses cannot be verified before join is executed). The optimization above will save time for the verification process.

5. Error Handling

5.1. Syntax Error

This is the most common type of error which includes all types of errors that do not follow MySQL syntax rules. Here are some examples:

```
-> CREATE TABLE rel1 (col1 INT, col2 INT, PRIMARY KEY (col1);
```
(missing right parenthesis)

```
-> INSERT rel1 VALUES (1,1);
```
(should be insert into)

5.2. Keyword Used

User input besides predefined keywords cannot be part of the keywords. For example, you cannot name a table 'table' because the system recognizes TABLE as a SQL keyword.

```
-> CREATE TABLE table (col1 INT, col2 INT, PRIMARY KEY (col1));
```

5.3. Invalid Type

Invalid Type is thrown when the type is not INT or STRING or the type being compared is not compatible. The following statements will throw this error:

```
-> CREATE TABLE course (course_num BIGINT, PRIMARY KEY (course_num));
-> SELECT * FROM rel1 WHERE col1 = 'dsa'; (assuming col1 is INT type)
-> SELECT * FROM employees JOIN departments on employees.emp_name =
departments.dep_id;
```

5.4. Duplicate Item

Duplicate Item is thrown when attributes are duplicated such as tables, primary keys, or columns:

```
-> CREATE TABLE rel1 (col1 INT, PRIMARY KEY (col1));
```

The above statement will throw an error if table rel1 already exists in a given instance of the DB.

```
-> INSERT INTO rel1 VALUES (1,2);
```

The above statement will throw an error if a tuple with a primary key value of 1 is already in table rel1.

```
-> CREATE TABLE course (course_num INT, course_num STRING, PRIMARY KEY
(course_num));
```

The above statement will throw an error due to the duplicate column name 'course_num'

5.5. NOT EXIST

Thrown when a user inputs a query that references attributes that do not exist, such as tables and columns:

```
-> SELECT * FROM rel5;
-> SELECT col3 FROM rel1;
```

5.6. Unsupported Functionality

Thrown for queries recognized in MySQL but are not supported in our system:

```
-> SELECT * FROM rel1,rel2;
-> SELECT MAX(col1), MAX(col2) FROM rel1;
```

Our system properly throws errors such that the program does not exit when encountering faulty user input. The program identifies the errors into the different types listed above (defined in exception.py) and provides comments to give users certain hints on where the error could be. It will then allow users to re-enter their input without quitting the program.

6. Constraints

6.1. Valid Input

- Only one valid and complete input can be processed at a time
- When inputting a query, users must end their query with a semicolon (;)

6.2. Foreign Key

- Foreign keys can only be defined once in each table
- A foreign key in a table has to have a 1-1 relation to the referenced table

6.3. EXECUTE

- Multiple queries can be processed sequentially if they are separated by semicolons in a SQL file, and executed using the keyword EXECUTE followed by the name of the sql file and a semicolon
- EXECUTE has no mechanism to ignore statements that are commented out in the input file. It simply reads all characters in the file and parses it by semicolon for further processing, so input files should not include comments

6.4. Tuples

- All attributes must not be NULL, errors will be thrown for incomplete tuples
- Assume we only need a primary indexing structure for key attributes (single or composite)